Test plan – Multi-Agentic RAG based AI Research Scientist (Team-44)

By – Sujato Dutta

1. Introduction

This document outlines the testing strategy and evaluation methodology for the Agentic RAG AI Research Scientist project. The testing is designed to automatically evaluate both the retrieval accuracy and the quality of the generated research syntheses.

2. Test Approach

The project uses an offline benchmark evaluation suite. Testing is fully automated through an evaluation runner script (evaluation/run_eval.py). The pipeline is evaluated end-to-end for a given set of queries.

3. Test Data

The evaluation relies on predefined datasets:

- Test Queries: A set of 15 predefined research queries stored in “test_queries.json”.
- Ground Truth Data: Expected relevant paper IDs for each query are maintained in “expected_papers.json”.

4. Test Execution

Tests are executed from the command line using the evaluation runner:

“python evaluation/run_eval.py”

5. Evaluation Metrics

The testing suite calculates metrics across two main categories to ensure both accurate retrieval and high-quality reasoning.

5.1 Retrieval Metrics

These metrics evaluate the effectiveness of the search and retrieval agents against the ground truth data:

- Recall@K: Measures the proportion of expected papers that were successfully retrieved in the top K results.
- Mean Reciprocal Rank (MRR): Evaluates the ranking quality of the retrieved papers.
- Context Precision: Measures the proportion of the retrieved papers that are actually relevant.

5.2 Reasoning and Quality Metrics (LLM-as-Judge)

The project uses a fast LLM model to judge and score the quality of the generated responses:

- Faithfulness Score (0.0 to 1.0): Evaluates if all claims in the generated response are supported by the retrieved evidence.
- Unsupported Claims Count: Counts the number of claims made that lack evidence.
- Hallucination Flag: A boolean flag detecting if information not present in the evidence was included.
- Synthesis Score (1 to 5): Evaluates how well the response integrates information from multiple papers.
- Comparison Score (1 to 5): Evaluates the depth of comparison and contrast between different approaches.
- Limitation Awareness Score (1 to 5): Evaluates how well the response acknowledges the limitations of the research.

5.3 System Metrics

- Total Latency: Measures the time taken to process the query from start to finish.
- Token Usage: Tracks the number of API tokens consumed during the execution.

6. Test Reporting

After running the evaluation, the system outputs an evaluation summary to the console containing the aggregate results (e.g., Success Rate, Average Recall, Average MRR, Hallucination Rate). These metrics are also logged in real time in Supabase backend for every user query to be monitored by the dev team in production and handle failures or bugs.